genfleet

GUC German University in Cairo الجامعة الألمانية بالقاهرة

Kestrel: Rogue-Agent Detection in a Multi-Tenant Agent Platform

Data Engineering · Winter 2026 · Semester project · Teams of 2–3 · Three milestones

Sponsored by GenFleet (genfleet.ai).

This is the project overview: the story, the data, and the rules that hold for the whole semester. What to build and submit for each milestone is in its own brief (`MILESTONE_1`, `MILESTONE_2`, `MILESTONE_3`). Field-level detail is in the companion **Dataset Description**.

1. Scenario

Kestrel is a fictional SaaS platform hosting autonomous AI agents. Each customer is a **tenant** (a bank, a hospital group, a logistics firm) running a small fleet of agents that plan tasks, call an LLM, and use tools (web fetch, SQL, shell, email, file I/O, HTTP). Every agent runs inside a **sandbox** with a declared policy: an allowlist of tools, an allowlist of network destinations, a filesystem scope, and a resource budget.

The platform promises agents cannot leave their sandbox. Occasionally that fails, and quietly:

> A marketing-research agent fetches a competitor's web page containing an **indirect prompt injection**. Its next turn silently adopts a new goal: enumerate the internal network, read whatever credentials it can reach, exfiltrate them. It stays inside its normal invocation loop the whole time. Nothing crashes. Every individual event looks ordinary.

The signal exists only **after integration**: the prompt turn where the stated goal changed, the tool call outside the allowlist, the flow outside the egress policy, the sandbox denial, and the SIEM alert from the targeted system are five records spread over four files in four formats, from three subsystems, with clock skew between them. (A **SIEM**, *security information and event management* system, is the tool a company's security team uses to collect logs from its own machines and raise alerts on suspicious activity. Here it belongs to the systems the rogue agent attacks, not to Kestrel.)

Your problem is not detection. It is building the data platform that makes detection possible. All data is synthetic: no production logs, credentials, personal data, or real companies. Attack techniques appear only as labels and MITRE ATT&CK identifiers (e.g. `T1046`); there are no working exploit payloads.

2. What you build

One pipeline, built in three milestones. Each closes on material the labs have already covered, so nothing is assessed before it is taught.

MILESTONE	PIPELINE STAGES	LABS	WEIGHT
Milestone 1	1. **Ingest and normalise** every format into one canonical activity model · 2. **Clean and integrate**: deduplicate, normalise timestamps, resolve entities, correlate across systems	1–3	30%

MILESTONE	PIPELINE STAGES	LABS	WEIGHT
Milestone 2	3. **Model and load** a PostgreSQL warehouse behind validated contracts · 4. **Stream** the live feed through Kafka · 5. **Analyse** in Spark, over deliberately skewed data	4–7	40%
Milestone 3	6. **Visualise** in an operational security dashboard · 7. **Ask**, through a read-only, injection-resistant agent	8–10	30%

By the end it must answer: which agents executed a tool outside their sandbox allowlist, and when did it start; which sessions show **goal drift** between the assigned task and the executed tool sequence; which forbidden egress destinations were contacted; the median time from the first suspicious prompt turn to the first policy violation; which agent versions are most susceptible to injection; how many sessions are unjoinable, duplicated, or incomplete, and why; and what each tenant spent on tokens.

3. Data you are given

Everything is in `kestrel-telemetry-pack-v1.zip`.

FILE	FORMAT	ROLE	FIRST USED
`tenants.csv`, `agents.csv`, `tools.csv`, `model_pricing.csv`	CSV	the companies, agent versions and metadata, tool catalogue, prices	M1
`sandbox_policies.json`	nested JSON	versioned tool allowlist, egress allowlist, FS scope, budgets	M1
`agent_activity.parquet`	Parquet	every session start (with the task it was *supposed* to do) and end, LLM turn (tokens, stated goal, injection indicators), and tool call (status, target)	M1
`network_flows.csv`	CSV	egress attempts, with verdict	M1
`sandbox_audit.log`	syslog text	runtime allow/deny decisions per syscall	M1
`siem_alerts.json`	nested JSON	what the *targeted* systems saw, with MITRE technique IDs	M1
`stream_events.json`	JSON	hold-out Kafka feed, terminated by an `EOS` control event	M2
`incident_labels.csv`	CSV	**held-out** ground truth: benign / suspicious / confirmed-rogue	M2

Default scale: 60 days, 6 tenants, 14 agents across 22 `(agent_id, release_version)` rows, 18 tools, ~100k sessions, ~940k activity rows, ~42k flows, ~184k sandbox decisions, ~9.6k stream events. Roughly **1.5% of sessions are rogue**. The class is rare by design, and every metric you report must survive that imbalance.

What to expect

This is an export from real-looking systems, and it has the problems real exports have. **Finding them is part of the work**: profile every file before you clean it, and document each problem you find, how you detected it, and what you decided to do about it.

- **Not everything is UTC.** The sources were written by different systems with different clocks. Work out each one's time convention, normalise everything to UTC, and say how you know.
- **No shared join key.** The SIEM feed knows nothing about Kestrel sessions. Linking its alerts to platform activity is your job; declare the rule you use and how often it fails.
- **Delivery is at least once.** Do not assume any record arrives exactly once, or on time.
- **Traffic is not evenly spread.** Find out where it concentrates, report it, and respond to it explicitly in Spark.

Every rejected record carries exactly one reason from a **closed enum**, so teams are comparable: `DUPLICATE_EVENT_ID`, `UNKNOWN_AGENT_ID`, `UNKNOWN_TENANT_ID`, `MALFORMED_TIMESTAMP`, `ORPHAN_EVENT`, `INVALID_EVENT_TYPE`, `MISSING_REQUIRED_FIELD`, `INVALID_NUMERIC_RANGE`, `LATE_BEYOND_WATERMARK`, `UNPARSEABLE_RECORD`, `SCHEMA_VIOLATION`.

Semantics you must not guess

- `event_time` is when it happened; `ingest_time` is when the platform received it.
- **Policy violation is derived** by joining an activity record to the policy version *effective at the record's* `event_time`, not the current one.
- **Session duration is derived**: terminal `event_time` − `session.start` `event_time`. Any `latency_ms` column is per-event and may only be used as a cross-check.
- `injection_indicators` is a noisy detector output, not truth. `incident_labels.csv` is truth, is sparse, and is **held out**. A rule that reads the label is a failed deliverable.
- `severity` in the SIEM feed is the target system's opinion, not yours.

4. Required metrics

Definitions are in Appendix A of the Dataset Description. State the denominator, population filter, late-event policy, and null policy for each.

#	METRIC	M1 (PANDAS)	M2 (WAREHOUSE AND SPARK)
1	Session volume, success rate, error rate	✓	✓
2	p50 / p95 session duration	✓	✓
3	Policy-violation and egress-violation rate		✓
4	Sandbox denial rate and escalation attempts		✓
5	Goal-drift and injection-indicator rate		✓
6	Tool failure and retry rate		✓
7	Blast radius and time-to-first-violation		✓
8	Trace completeness and correlation coverage	✓	✓

#	METRIC	M1 (PANDAS)	M2 (WAREHOUSE AND SPARK)
9	Rejected-record and late-event rate	✓	✓
10	Detection precision / recall against `incident_labels.csv`		✓
11	Estimated LLM cost at the price effective at turn time		✓

Declare whether a missing token count is null-excluded or zero, and show the effect on at least one metric. Report each value with the policy behind it. Marking compares your values with reference values computed by the course staff, allowing for differences you have documented.

5. Marking and rules

Weights: **M1 30% · M2 40% · M3 30%.** The last page of each milestone brief gives its grading scheme.

Every milestone requires a reproducible `docker compose up` from a clean clone. **Every member must be able to explain any part of the pipeline**.

The pack contains no exploit code; do not reproduce any described technique against real infrastructure. Library reuse is expected and should be cited; submitting another team's pipeline is not.

6. Submission

- **One** team member creates a **private GitHub repository** named `<teamname>_agents` (for example `quants_agents`) and adds the other team members and `@de-guc` as collaborators. No other member creates a copy.
- The repository has one folder per milestone: `M1/`, `M2/`, and `M3/`. Each milestone's work goes in its own folder.
- Submit the repository link through the Google Form: https://forms.gle/q8y1NWYBv4Rbv9989.
- **Individual work is assessed from each member's commits.** Commit your own work, from your own account.